CE-355 Lab 1
TDS Monitor

1. Introduction

The lab will introduce the basic FPGA development workflow:

```
Design → Simulate → Synthesize → Program → Test
```

In this lab, you will design and simulate a simple Total Dissolved Solids (TDS) monitoring system using the DE25-Standard FPGA development board. The system accepts a digital measurement from an Analog-to-Digital Converter (ADC), converts the measurement into a TDS value, displays the result on the seven-segment displays, and activates an LED when the measured TDS exceeds a specified threshold.

The primary focus of this lab is structural VHDL design. Rather than implementing the entire system as a single block of logic, you will construct the system by connecting several pre-defined functional components. Each component performs a specific task, and the top-level design integrates these components to form a complete system.

The ADC, seven-segment displays, and LED are treated as external interfaces to the TDS-monitoring system. This allows the design to be simulated using digital input and output signals without modeling the physical sensor, ADC hardware, or display hardware.

By completing this lab, you will:

- Construct a hierarchical FPGA design by instantiating and connecting VHDL components.
- Use VHDL entities, architectures, signals, component instantiation, and port map statements to create a structural design.
- Implement and integrate combinational hardware modules for TDS conversion, binary-to-BCD conversion, and seven-segment display control.
- Develop a simulation testbench to verify the behavior of the complete design using digital inputs and outputs.
- Compile, synthesize, and program the design using Altera Quartus Prime and the DE25-Standard FPGA board.

2. Background

A Total Dissolved Solids (TDS) sensor estimates the concentration of dissolved substances, such as salts and minerals, in water by measuring its electrical conductivity. When dissolved substances separate into electrically charged ions, they allow water to conduct electricity. The sensor measures this conductivity and produces an analog output voltage related to the concentration of dissolved substances. It uses an AC excitation signal to measure conductivity

while helping prevent electrode polarization. Unlike a turbidity sensor, which measures suspended particles that affect water clarity, a TDS sensor estimates dissolved substances that may be present even in perfectly clear water.

2.1 Analog-to-Digital Conversion

The sensor produces an analog voltage that the FPGA cannot process directly. An analog-to-digital converter (ADC) converts this voltage into a digital value that the FPGA can use. The DE25-Standard interfaces with the analog signal through its ADC hardware, which samples the input voltage and produces a digital representation of the measurement.

In this lab, the ADC value is represented by a 12-bit input, providing 4,096 possible values ranging from 0 to 4,095. Assuming an input voltage range of 0 V to 3.3 V, the relationship between voltage and the digital ADC value is approximately:

$$ADC = \frac{V_{in}}{3.3} \times 4095$$

The input voltage can also be estimated from the ADC value:

$$V_{in} = \frac{ADC}{4095} \times 3.3$$

For example, an input voltage of approximately 0.103 V corresponds to an ADC value of 128, represented in hexadecimal as X"080".

2.2 Converting ADC Values to TDS

The FPGA uses the ADC value to estimate the TDS concentration in parts per million (ppm). The `tds_converter` module performs this conversion using a simplified approximation of the reference calibration curve published for the DFRobot Gravity TDS sensor at 25°C. Because this lab uses a HiLetgo TDS sensor, the reference curve has not been verified for the specific sensor being used. **Calibration is required to establish how accurately the calculated TDS values correspond to actual measurements from the TDS sensor.**

Rather than implementing the complete mathematical equation, the converter divides the ADC input range into eight segments. Each segment has a starting TDS value and an approximate conversion rate. The three most significant ADC bits select the segment, while the remaining nine bits determine the offset within that segment. The converter uses addition and right shifts to calculate the estimated TDS value efficiently in hardware.

For example, an ADC value of 128 (`X"080"`) falls within the first segment. The converter calculates the estimated TDS as follows:

$$TDS = \frac{128}{4} + \frac{128}{16} = 32 + 8 = 40 \text{ ppm}$$

Therefore, an ADC input corresponding to approximately 0.103 V produces an estimated TDS reading of 40 ppm in the current implementation. This is a calculated estimate, not a verified measurement of the water's actual TDS concentration.

To improve accuracy, the TDS sensor should be calibrated using water samples or standard solutions with known TDS concentrations. The corresponding sensor output voltages can then be measured and compared with the FPGA's calculated values. These measurements can be used to adjust the conversion algorithm to better match the sensor's actual response.

Until calibration is performed, the displayed TDS values should be treated as estimates based on the reference curve rather than precise measurements. The LED threshold detection still demonstrates how the FPGA compares a calculated TDS value against a configured limit, but its response to actual water samples will depend on the accuracy of the conversion.

2.3 Threshold Detection

The tds_converter module also compares the estimated TDS reading against a configurable threshold. When the reading exceeds the threshold, the converter activates its threshold output, which can control an LED in the tds_monitor design.

For this lab, the intended threshold is 250 ppm. The LED should remain off when the estimated TDS is at or below 250 ppm and turn on when the reading exceeds 250 ppm. This provides a simple way to monitor water quality using the sensor, ADC, and FPGA.

3. Project Setup

You will use **ModelSim** to simulate and verify your designs. You will use **Altera Quartus Prime** to compile, synthesize, and program your design on the DE25-Standard FPGA development board.

Maintaining an organized file structure is an important part of RTL design. Each directory has a specific purpose. For each lab, organize your project using the following directory structure:

```
Lab 1/
│
├── vhdl/   – Contains the VHDL source files for the design.
├── sim/    – Contains the ModelSim testbench and simulation files.
└── syn/    – Contains the Altera Quartus Prime synthesis files.
```

*NOTE: Throughout this assignment, please refer to the **DE25-Standard Development Kit User Manual** and the **DE25-Standard Getting Started Guide,** which contains more details on the board set up, pinouts, and configuration.*

4. System Design

The TDS monitor is constructed from several independent hardware modules connected together to form a complete system. This approach is known as structural design. Each module

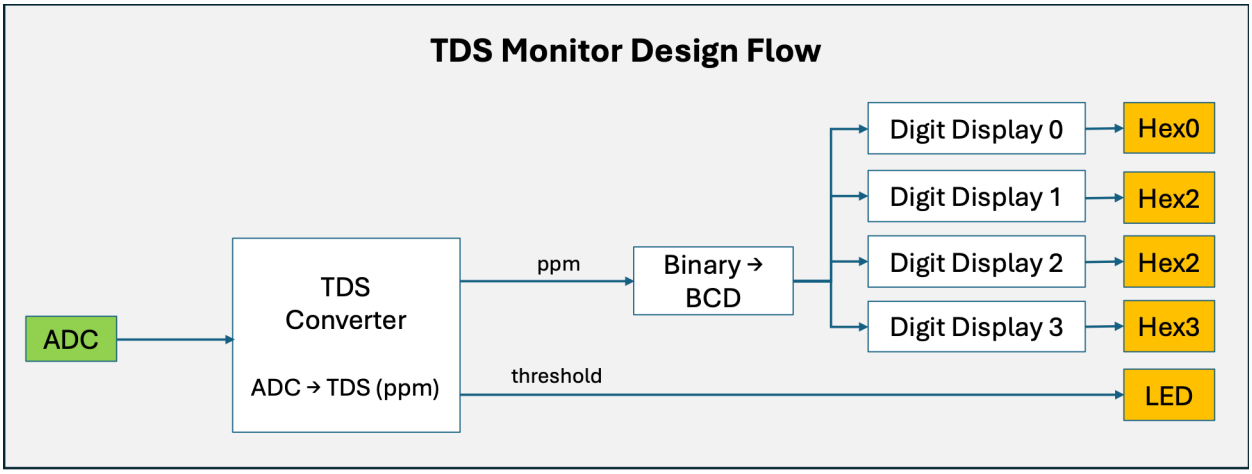
performs a specific function, while the top-level `tds_monitor` entity connects the modules and routes signals between them. The overall system is organized as follows:

The physical peripherals are treated as external interfaces to the design. The ADC provides a 12-bit digital input representing the analog voltage produced by the TDS sensor, while the seven-segment displays and LED receive digital outputs from the system. This allows the core functionality to be simulated using digital signals without modeling the physical sensor, ADC, or display hardware.

The system consists of three functional modules and a top-level system design:

- **TDS Converter** — Receives the ADC value, converts the measurement into an estimated TDS value, and determines whether the measured TDS exceeds the specified threshold. It outputs both the calculated TDS value and the threshold signal for the LED.
- **BCD Converter** — Converts the selected 8-bit TDS value into separate hundreds, tens, and ones decimal digits.
- **7-Segment Display Decoder** — Converts each 4-bit digit into the corresponding seven-segment display pattern.
- **Top-Level TDS Monitor** — Connects the functional modules together into a structural system design.

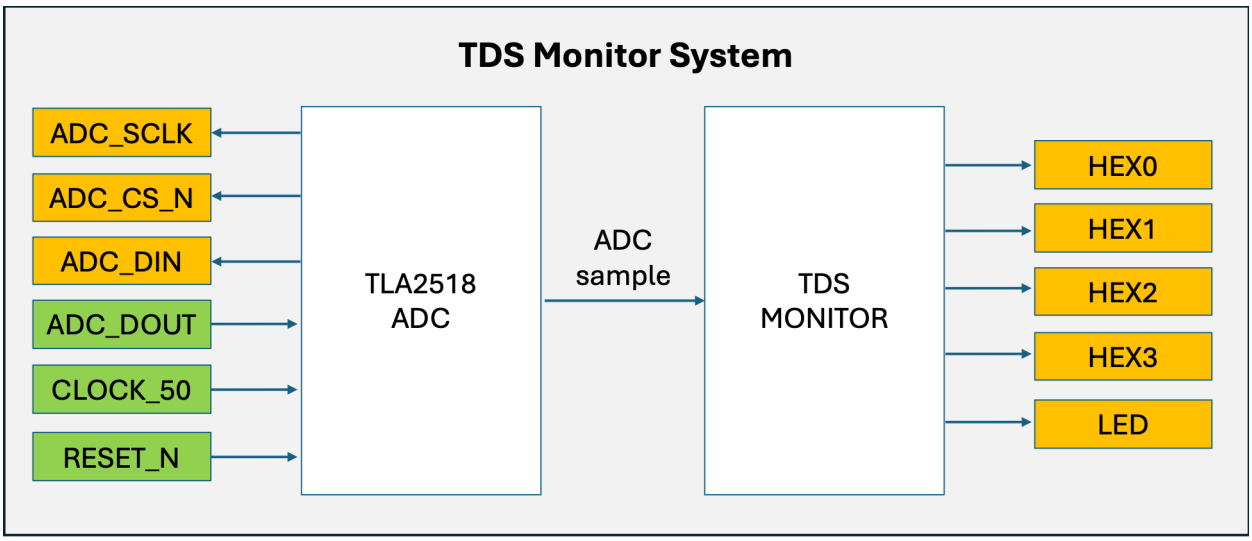
The top-level `tds_monitor` entity integrates these modules. The ADC value is provided to the TDS converter, which calculates the estimated TDS value and determines whether the threshold has been exceeded. The resulting TDS value is passed to the BCD converter, which produces the 4 decimal digits (thousands, hundreds, tens, and ones). The top-level design also connects the display decoders to the appropriate digit signals. The threshold output from the TDS converter is connected directly to an LED, which is enabled if the ppm exceeds a designated threshold value.



The top-level design does not perform the TDS conversion, binary-to-BCD conversion, or seven-segment decoding itself. Instead, it instantiates the appropriate components and connects their inputs and outputs in a structural design. This separation allows each module to be developed

and verified independently and provides a modular structure that can be reused in future designs.

Important: The complete hardware design includes a provided TLA2518 ADC interface and `tds_monitor_system` entity, which connects `tds_monitor` to the onboard ADC. This system-level design is used only during synthesis. The ADC is not simulated because its SPI interface requires clocked logic, which is outside the scope of this introductory combinational-design exercise. Instead, `tds_monitor` is simulated directly, with the ADC represented as a 12-bit digital input driven by the testbench. During synthesis, `tds_monitor_system` connects the design to the physical onboard ADC.



5. Design Implementation

In this section, you will complete the `tds_monitor` architecture by instantiating the provided components and connecting them together. The design should be implemented using structural VHDL. Do not use processes in this section.

The provided `tds_converter`, `bcd`, and `digit_display` entities define the functionality of the individual components. Your task is to connect these components to create the complete TDS monitor.

5.1 Component Declarations

The first step is to declare the components that will be used in the structural design. Add component declarations in `tds_monitor` for the modules used in the design. The component declarations should match the corresponding entity declarations provided with the lab. Pay particular attention to the port names, directions, and signal widths.

```
component digit_display is
    port(
        din  : in std_logic_vector(3 downto 0);
        dout : out std_logic_vector(6 downto 0)
    );
end component digit_display;

component bcd is
    port(
        bin       : in std_logic_vector(7 downto 0);
        thousands : out std_logic_vector(3 downto 0);
        hundreds  : out std_logic_vector(3 downto 0);
        tens      : out std_logic_vector(3 downto 0);
        ones      : out std_logic_vector(3 downto 0)
    );
end component bcd;

component tds_converter is
    port(
        adc         : in  std_logic_vector(11 downto 0);
        tds         : out std_logic_vector(7 downto 0);
        threshold   : out std_logic
    );
end component tds_converter;
```

Note that the `tds_converter` has two generic parameters. You can ignore these for now, and they don't need to be included in the component declaration. We will use generic parameters in future lab assignments.

5.2 Signal Declarations

The components must be connected using internal signals. Add the following signals to the architecture:

Signal	**Width**	**Purpose**
tds	8 bits	Calculated TDS value from tds_converter
digit0	4 bits	Ones digit
digit1	4 bits	Tens digit
digit2	4 bits	Hundreds digit
digit3	4 bits	Thousands digit

Use `std_logic_vector` for all five signals. The resulting declarations should follow the form:

```
signal tds         : std_logic_vector(7 downto 0);
signal digit0      : std_logic_vector(3 downto 0);
signal digit1      : std_logic_vector(3 downto 0);
signal digit2      : std_logic_vector(3 downto 0);
signal digit3      : std_logic_vector(3 downto 0);
```

5.3 Instantiate the TDS Converter

The tds_converter is responsible for converting the ADC measurement into the selected TDS unit and determining whether the TDS exceeds the threshold.

Create an instance of the component and connect:

- **adc** port to the top-level **adc** input.
- **tds** port to the internal **tds** signal.
- **threshold** port directly to the top-level **led** output.

The threshold output does not need an intermediate signal because the LED is simply the output of the TDS converter.

```
tds_converter_inst : tds_converter
port map(
    adc         => adc,
    tds         => tds,
    threshold   => led
);
```

5.4 Instantiate the BCD Converter

The selected TDS is represented as an 8-bit binary value. The 7-segment displays, however, require individual decimal digits.

The provided `bcd` component converts the binary TDS into three 4-bit decimal digits. Instantiate the `bcd` component and connect the ports to local signals as follows. Note that `digit0` is not connected to the BCD converter. It is used for the TDS unit.

```
bcd_inst : bcd
    port map(
        bin       => tds,
        thousands => digit3,
        hundreds  => digit2,
        tens      => digit1,
        ones      => digit0
    );
```

5.5 Instantiate the 7-Segment Display Decoders

The final step is to connect each 4-bit digit to a digit_display component. Instantiate four copies of digit_display. Connect the components as follows:

Display	Input signal	Function
`HEX0`	`digit0`	Ones
`HEX1`	`digit1`	Tens
`HEX2`	`digit2`	Hundreds
`HEX3`	`digit3`	Thousands

Each instance should connect its 4-bit `din` input to the appropriate digit signal and its 7-bit `dout` output to the corresponding top-level display output, for example:

```
display0 : digit_display
port map(
    din  => digit0,
    dout => hex0
);
```

6. Simulation

Before programming the FPGA, you will verify your design using ModelSim. Simulation allows you to test the behavior of the tds_monitor without connecting the physical ADC, 7-segment displays, or LED.

For this lab, the testbench is provided. You will complete the testbench by connecting the device under test, adding the required output signals, and creating additional test cases.

The testbench supplies digital values to the tds_monitor and observes its digital outputs. The physical peripherals are not modeled.

The provided testbench contains four basic sections, as follows.

1. Local signal declarations
2. Component declarations
3. Device Under Test (DUT) instantiation
4. Test stimulus process

6.1 Add Signal Declarations

The testbench already contains signals for the two inputs:

```
signal adc : std_logic_vector(11 downto 0);
```

Add signals for the five outputs of the tds_monitor, which allows ModelSim to observe the outputs of the design.

```
signal hex0 : std_logic_vector(6 downto 0);
signal hex1 : std_logic_vector(6 downto 0);
signal hex2 : std_logic_vector(6 downto 0);
signal hex3 : std_logic_vector(6 downto 0);
signal led  : std_logic;
```

6.2 Instantiate the Device Under Test

The tds_monitor component is the Device Under Test (DUT). Instantiate it in the architecture and connect each port to the corresponding local testbench signal. The completed instance should have the following structure:

```
dut : tds_monitor
port map(
    adc  => adc,
    hex0 => hex0,
    hex1 => hex1,
    hex2 => hex2,
    hex3 => hex3,
    led  => led
);
```

6.3 Understand the Test Stimulus

The test stimulus is implemented using a VHDL process. The process changes the input signals and then waits for a specified amount of simulation time:

```
-- 0.103 V : 0040 ppm, LED off (distilled / RO water)
adc <= X"080";
wait for 20 ns;
```

The **wait for 20 ns** statement allows the design to respond before the next test case is applied. Although the tds_monitor design does not use processes, processes are appropriate in a testbench because the testbench must generate a sequence of input events over time.

ADC values in the testbench should be specified using hexadecimal notation for easier comprehension and bit alignment. For example, the value `0x080` is decimal 128. With the conversion used in this lab, this represents: V_{in} = 128 x 3.3 / 4095 = 0.103 V.

6.4 Add Test Cases

Extend the provided stimulus process with at least 10 additional test cases. Your test cases should verify a range of ADC input values and the corresponding TDS estimates and LED states. At a minimum, include tests for:

- Several TDS levels below the threshold.
- A TDS level exactly at the threshold.
- A TDS level above the threshold.
- A low ADC value representing a low input voltage.
- A high ADC value representing a high input voltage.
- The minimum ADC input value.
- The maximum ADC input value.
- Several intermediate ADC values.
- Increasing the ADC value across multiple test cases.
- Changing the ADC value from above the threshold to below it.

For each test case, specify the expected ADC value, approximate input voltage, estimated TDS level, and LED state before running the simulation.

For example:

```
-- Test case: Approximately 0093 ppm
-- ADC = 300, input voltage approximately 0.242 V
-- Expected LED state: OFF
adc <= X"12C";
wait for 20 ns;

-- Test case: Approximately 0250 ppm
-- ADC = 835, input voltage approximately 0.673 V
-- Expected LED state: OFF
adc <= X"343";
wait for 20 ns;

-- Test case: Approximately 0251 ppm
-- ADC = 836, input voltage approximately 0.674 V
-- Expected LED state: ON
adc <= X"344";
wait for 20 ns;
```

Observe the hexadecimal ADC values and the resulting seven-segment display outputs (`hex0` through `hex3`) and LED output in the simulation waveform.

Compare the actual outputs against your expected results. Confirm that the displayed TDS estimate changes as the ADC input changes and that the LED turns on only when the estimated TDS exceeds the configured threshold.

Remember that the ADC value represents the input voltage, not the TDS level itself. The TDS estimate depends on the conversion implemented in `tds_monitor`, so your expected results should match the conversion used by your design.

6.5 Create the ModelSim Command File

We will create a ModelSim command file to compile and run the simulation. Create a new folder at the top level of your project directory called **sim**, and inside that directory create a file named **`sim.do`**. Then, using a text editor, enter the following commands and save the file:

```
vlib work

vcom -2008 ../vhdl/digit_display.vhd
vcom -2008 ../vhdl/bcd.vhd
vcom -2008 ../vhdl/tds_converter.vhd
vcom -2008 ../vhdl/tds_monitor.vhd
vcom -2008 ../vhdl/tds_monitor_tb.vhd

vsim -voptargs=+acc tds_monitor_tb

add wave -group "TESTBENCH" sim:/tds_monitor_tb/*
add wave -group "TDS MONITOR" sim:/tds_monitor_tb/dut/*
add wave -group "TDS CONVERTER" sim:/tds_monitor_tb/dut/tds_converter_inst/*
add wave -group "BCD" sim:/tds_monitor_tb/dut/bcd_inst/*

run 1000 ns
```

The commands perform the following operations:

- `vlib work` creates the ModelSim working library.
- `vcom` compiles each VHDL source file. `-voptargs=+acc` ensures that ModelSim doesn't optimize away the internal logic so you can debug intermediate signals.
- `vsim` starts the `tds_monitor_tb` simulation.
- `add wave` adds signals to the waveform viewer.
- `run` executes the testbench for the specified simulation time.

5.6 Run the Simulation

To run the simulation, open Modelsim and change to the **sim** directory where the simulation will be compiled. In the command window, type: **do sim.do** and hit enter to run the simulation.

You can also launch the modelsim simulation directly from a terminal by running:

vsim -do sim.do

If the design compiles successfully, ModelSim will display the testbench waveforms. Any errors will appear in the console log window. Examine the waveforms and verify that the outputs correspond to the input values and `SW0` settings.

Pay particular attention to:

- `HEX3`, `HEX2`, `HEX1`, and `HEX0` displaying the selected TDS.
- `LED` changing state when the Fahrenheit TDS crosses the threshold.

6. Synthesis

Once the design has been successfully simulated, it can be implemented on the DE25-Standard. In this stage, you will use **Altera Quartus Prime** to compile and synthesize the design, assign the FPGA pins, program the board, and verify the system using the TDS sensor.

6.1 Create the Quartus Prime Project

Create a new Quartus Prime project in the **syn** directory of your lab project called **tds_monitor_system**. Add the following VHDL source files from the **vhdl** directory:

- tds_monitor_system.vhd
- tds_monitor.vhd
- tds_converter.vhd
- digit_display.vhd
- bcd.vhd
- tla2518_adc.vhd

Make sure to set **tds_monitor_system** as the top-level entity. Verify that the project is configured for the Agilex-5 FPGA device **A5ED013BB32AE4SR1** used on the DE25-Standard. Refer to the DE25-Standard manual for the appropriate device and project settings.

6.2 Assign FPGA Pins

The top-level signals of the tds_monitor_system must be assigned to the physical FPGA pins connected to the peripherals. To assign the pins in Quartus, from the menu navigate to Assignment > Pin Planner. You may not be able to add the pin successfully if the FPGA device has not been assigned to the project. Use the list below and DE25-Standard manual to assign the appropriate FPGA pins for:

- HEX0 through HEX3
- LED
- CLOCK
- RESET_N
- ADC

Port	Pin	Description
LED	CC71	LED [0]
CLK	D8	50-MHz FPGA clock
RESET_N	PB9	Active-low FPGA reset
ADC_CONVST	BP22	Conversion Start
ADC_DOUT	CK4	Digital data input
ADC_DIN	BF32	Digital data output
ADC_SCLK	CH4	Digital clock input
HEX0[0]	BP81	Seven Segment Digit 0[0]
HEX0[1]	BM81	Seven Segment Digit 0[1]
HEX0[2]	BM89	Seven Segment Digit 0[2]
HEX0[3]	BK89	Seven Segment Digit 0[3]
HEX0[4]	BH92	Seven Segment Digit 0[4]
HEX0[5]	BM92	Seven Segment Digit 0[5]
HEX0[6]	BP92	Seven Segment Digit 0[6]
HEX1[0]	CA78	Seven Segment Digit 1[0]
HEX1[1]	BW78	Seven Segment Digit 1[1]
HEX1[2]	BU78	Seven Segment Digit 1[2]
HEX1[3]	BR78	Seven Segment Digit 1[3]
HEX1[4]	BU81	Seven Segment Digit 1[4]
HEX1[5]	BR81	Seven Segment Digit 1[5]
HEX1[6]	CA89	Seven Segment Digit 1[6]

Port	Pin	Description
HEX2[0]	BW89	Seven Segment Digit 2[0]
HEX2[1]	BU92	Seven Segment Digit 2[1]
HEX2[2]	BR92	Seven Segment Digit 2[2]
HEX2[3]	BU89	Seven Segment Digit 2[3]
HEX2[4]	BR89	Seven Segment Digit 2[4]
HEX2[5]	CF78	Seven Segment Digit 2[5]
HEX2[6]	CH78	Seven Segment Digit 2[6]
HEX3[0]	CC81	Seven Segment Digit 3[0]
HEX3[1]	CA81	Seven Segment Digit 3[1]
HEX3[2]	CH81	Seven Segment Digit 3[2]
HEX3[3]	CF81	Seven Segment Digit 3[3]
HEX3[4]	CF89	Seven Segment Digit 3[4]
HEX3[5]	CH89	Seven Segment Digit 3[5]
HEX3[6]	CH92	Seven Segment Digit 3[6]
HEX4[0]	CF92	Seven Segment Digit 4[0]
HEX4[1]	CA92	Seven Segment Digit 4[1]
HEX4[2]	CC92	Seven Segment Digit 4[2]
HEX4[3]	CL76	Seven Segment Digit 4[3]
HEX4[4]	CK76	Seven Segment Digit 4[4]
HEX4[5]	CL82	Seven Segment Digit 4[5]
HEX4[6]	CK80	Seven Segment Digit 4[6]

6.3 Compile and Synthesize

Run the Quartus Prime compilation flow. The compilation should complete without errors. Review the compilation messages and correct any errors before continuing. Warnings should also be reviewed to determine whether they indicate a problem with the design.

Verify that Quartus recognizes **tds_monitor_system** as the top-level entity and that the expected component hierarchy is present. After compilation, review the Quartus Prime reports and verify that:

- The design compiles without errors.
- All required top-level signals have valid pin assignments.
- The expected hierarchy is present.
- No unexpected unconnected or undriven signals are reported.
- The design successfully completes synthesis and fitting.
- Timing analysis does not report critical violations.

The TDS monitor is a combinational design and does not contain a clocked process (other than the ADC). Therefore, there are no clock-based setup and hold requirements to verify for the TDS-processing logic. Timing reports should still be reviewed for unexpected warnings or other issues.

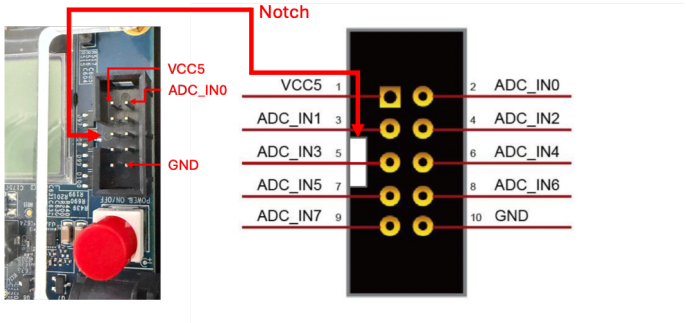
6.5 Program the FPGA

Once the design successfully compiles, use the DE25-Standard's **USB-Blaster** connection to program the FPGA. Open the Quartus Prime Programmer, select the appropriate programming file, and program the device. Refer to the DE25-Standard manual for information about connecting the board and using the USB-Blaster. After programming, the FPGA will begin operating with the synthesized TDS-monitoring design.

7. Hardware Setup and Testing

The lab kit includes an analog TDS sensor and a 3-pin JST-PH cable. The TDS produces an analog voltage proportional to TDS, which is connected to the DE25-Standard's onboard TLA2518 ADC. Use the additional M-F cable to connect the JST-PH cable to the correct pinouts on the FPGA board as specified below. Connect the TDS as follows:

TDS	DE25-Standard
Red — Power	3.3 V (VCC)
Black — Ground	GND
White — Analog Output	ADC_IN0



⚠ **IMPORTANT: Turn off the DE25-Standard before making connections. Make sure the notches are aligned as depicted above, and that you connect the correct pins to VCC and GND, or you will burn out the TDS chip.**

After making the connections, power on the DE25-Standard, and **reprogram the FPGA** before testing. Verify the operation of the complete system:

- Verify that the TDS digits are displayed on `HEX0`–`HEX3`.
- Dip the TDS sensor in distilled water and salt water to see the values change.
- Verify that the LED activates when the TDS exceeds the specified threshold.

Compare the hardware results with the behavior observed during ModelSim simulation. The same digital TDS-processing logic should produce the same results for the same ADC input.

The physical sensor introduces additional effects that are not present in simulation, including sensor accuracy and changes in TDS over time and at different temperatures. These differences are expected.

8. Submission

Zip your folder, name it as **CE355_Lab1_Team_[N].zip** and submit your code to Canvas. Make sure to clean out any synthesis and simulation work directories.

You will also need to demo the hardware set up and simulation during TA office hours. Be prepared to explain how the system works, simulation results, and behavior of the hardware system.